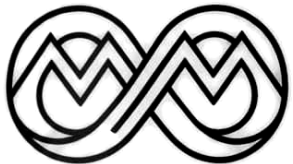


MLOPS MENA COMMUNITY PRESENTS

MLflow

The AI Engineering Platform

Experiment Tracking • Model Registry • LLMs, Agents & RAG • MLflow vs. Weights & Biases



MLOps MENA Community



What We'll Cover Today

1

Why MLflow?

The chaos it fixes, and the one idea behind it.

2

Platform Overview

Where classic ML and GenAI meet inside one tool.

3

Experiment Tracking

Runs, models, autologging, and the tracking environment.

4

Model Registry

From Staging to Production, with full lineage.

5

MLflow for LLMs, Agents & RAG

Tracing, evaluation, prompts, and the AI Gateway.

6

MLflow vs. Weights & Biases

Picking the right tool for your workflow.

The Problem Before MLflow

1

model_v2_final_FINAL.pkl

Dozens of near-identical files, and nobody remembers which one is actually correct.

2

Metrics scattered everywhere

Numbers live in notebooks, spreadsheets, or nowhere at all.

3

“What was this model trained on?”

After a few experiments, the parameters and data behind a model become a mystery.

4

Going back is painful

If an older model turns out to be better, reproducing it is a headache.

What Is MLflow?

- MLflow is the largest open source AI engineering platform for agents, LLMs, and traditional ML models.
- It packages every experiment as one unit: parameters, metrics, artifacts, and the model itself.
- Instead of asking “where did this model come from?”, you open the run and see the whole story.
- It works the same way whether you're training a scikit-learn classifier or shipping a production RAG agent.

One package, every time

Parameters

+ Metrics

+ Artifacts

+ Model

= a single, traceable MLflow Run.

Before vs. After MLflow

Before

- Copies of models with unclear names and versions
- Metrics recorded inconsistently, if at all
- No link between a model and the code/data that produced it
- Reproducing a past result means guessing

After

- Every run is logged as one traceable unit
- Metrics, parameters, and artifacts live together
- Full lineage from data → run → model
- Any past model version can be reloaded on demand

THE CORE IDEA

MLflow doesn't make your model better — it makes your ML workflow traceable, reproducible, and easy to go back to.

— the whole point of experiment tracking

MLflow at a Glance

30M+

Monthly downloads across the
MLflow ecosystem

100+

Framework and LLM provider
integrations

26K+

GitHub stars, as part of the Linux
Foundation

OTel

Fully OpenTelemetry-compatible
tracing, with native GenAI
semantic conventions

Open source and vendor-neutral — self-hosted, on-prem, or via managed platforms like Databricks, SageMaker, and Azure ML.

One Platform, Two Worlds

1

Classic ML & Deep Learning

Experiment Tracking, MLflow Model format, Datasets, Model Registry, Evaluation, and Deployment for scikit-learn, XGBoost, PyTorch, TensorFlow, Keras, Spark MLlib and more.

2

Agents, LLMs & RAG

Tracing (observability), Evaluation & Monitoring, Prompt Registry & Optimization, and the AI Gateway for managing cost and access to LLM providers.

3

Shared foundation

Both sides sit on the same Tracking Server, backend store, and artifact store — one platform to run, not two.

4

Runs anywhere

Local machine, on-prem clusters, Kubernetes, or managed services — Databricks, AWS SageMaker, Azure ML, Red Hat OpenShift AI, Nebius.

02

Experiment Tracking

Log parameters, metrics, and artifacts — then compare, visualize, and reproduce.



Three Core Concepts

1

Runs

A single execution of your ML code (e.g. one python train.py). Each run records metadata — metrics, parameters, timestamps — and artifacts such as model weights or images.

2

Models

The trained artifacts produced during a run. Logged Models carry their own metadata and artifacts, just like runs do.

3

Experiments

A container that groups runs and models around one task. Create and search experiments from the CLI, API, or UI.

Log Your First Run



python

```
import mlflow

with mlflow.start_run():
    mlflow.log_param("lr", 0.001)

    # Your ML code
    ...

    mlflow.log_metric("val_loss", val_loss)
```

mlflow.start_run() opens a run; log_param() and log_metric() record what happened inside it.

Autologging — Zero Manual Log Statements



python

```
import mlflow

mlflow.autolog()

# Your training code...
# params, metrics, and the model are captured automatically
```

Supported out of the box: scikit-learn, XGBoost, PyTorch, Keras, Spark, and more.

Log Model Checkpoints During Training



python

```
with mlflow.start_run() as run:
    for epoch in range(100):
        train_model(model, epoch)

        if epoch % 10 == 0:
            model_info = mlflow.pytorch.log_model(
                pytorch_model=model,
                name=f"checkpoint-epoch-{epoch}",
                step=epoch,
            )
```

Each checkpoint gets its own unique model ID — perfect for deep learning workflows.

Link Metrics to Models & Datasets



python

```
train_dataset = mlflow.data.from_pandas(train_df, name="training_data")

mlflow.log_metric(
    key="f1_score",
    value=0.95,
    step=epoch,
    model_id=model_info.model_id,    # links to a specific checkpoint
    dataset=train_dataset,           # links to a specific dataset
)
```

Full traceability: which model checkpoint, trained on which dataset, hit which score.

Search & Rank Logged Models



python

```
top_models = mlflow.search_logged_models(  
    experiment_ids=["1", "2"],  
    filter_string="metrics.accuracy > 0.95 AND params.model_type = 'RandomForest'",  
    order_by=[{"field_name": "metrics.f1_score", "ascending": False}],  
    max_results=5,  
)  
  
best = top_models[0]  
loaded_model = mlflow.pyfunc.load_model(f"models:{best.model_id}")
```

SQL-like filtering across metrics, params, and attributes — then load the winner directly.

The New Model URI Format (MLflow 3)

1

models:/<model_id>

Direct model reference — no need to know the run ID or the artifact path inside it.

2

models:/<model-name>/<model-version>

For models registered in the Model Registry, reference by name and version instead.

3

Replaces runs:/<run_id>/<artifact_path>

The older run-based URI still works, but the new format gives cleaner lineage between models, metrics, and datasets.

Tracking Datasets & the Tracking UI

- `mlflow.log_input()` records the datasets associated with a training run, alongside its metadata.
- The Tracking UI lets you visually explore experiments, runs, and models in one place.
- Compare runs — even across multiple experiments — side by side.
- Search runs by parameter or metric value, visualize metrics over time, and download artifacts.

Launch the UI

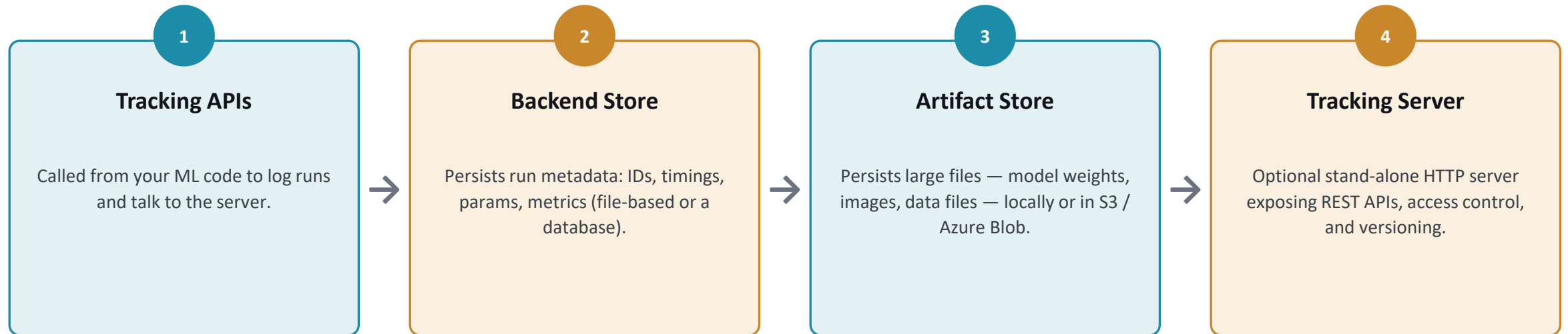
```
mlflow server --port 5000
```

Then open:

```
http://127.0.0.1:5000
```

(or your remote Tracking Server's address)

Setting Up the Tracking Environment



Three Common Setups

Setup	Scenario	Use Case
Localhost (default)	Solo development	Runs logged to a local mlruns directory — zero setup required.
Local DB backend	Solo development	A SQLAlchemy-compatible database (SQLite, PostgreSQL, MySQL) for cleaner metadata management.
Remote Tracking Server	Team development	A shared server proxies artifact and metadata requests, with proper access control for the whole team.

Keeping Many Runs Organized

1

Organize runs into experiments

Experiments are logical containers you can create via the CLI, API, or UI.

2

Create child runs

Group related runs under one parent — e.g. one child run per fold in cross-validation.

3

Tag your runs

Attach arbitrary key-value tags to filter and search runs later.

4

Run in parallel

MLflow supports launching multiple runs at once via multiprocessing or threading.

03

Model Registry

A centralized store for versioning, aliasing, and governing the full model lifecycle.



Why a Model Registry?

1

Version control

Automatically tracks every version, so teams can compare iterations, roll back, or run staging and production in parallel.

2

Lineage & traceability

Every registered version links back to the run or notebook that produced it — fully reproducible.

3

Production-ready workflows

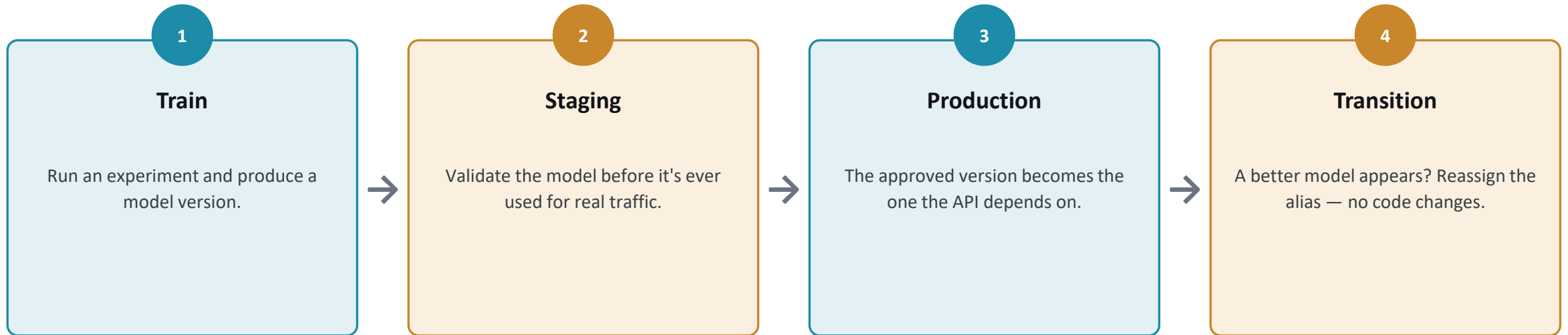
Aliases (like @champion) and tags make promoting a model to staging or production controlled and auditable.

4

Governance & compliance

Structured metadata, tagging, and role-based access support enterprise-grade governance requirements.

From Staging to Production



The API asks for the “champion” model by name and stage — never a hardcoded file path like `/models/model_17.pkl`.

Core Concepts

Concept	What It Means
Registered Model	A model with a unique name, containing versions, aliases, tags, and metadata.
Model Version	Each new registration under the same name increments the version number (v1, v2, ...).
Model URI	Reference format models:/<model-name>/<model-version>, e.g. models:/MyModel/1.
Model Alias	A mutable, named pointer to a version — e.g. @champion — used via models:/MyModel@champion.
Tags	Key-value labels on models or versions, e.g. validation_status: approved.
Annotations	Markdown descriptions covering methodology, datasets used, or algorithm notes.

Register & Load a Model



python

```
# Option 1 – register while logging
mlflow.<flavor>.log_model(..., registered_model_name="MyModel")

# Option 2 – register an already-logged model
mlflow.register_model(model_uri="<YOUR_MODEL_URI>", name="MyModel")

# Load it back by name and version
mlflow.<flavor>.load_model("models:/MyModel/1")
```

Two ways in, one way out — loading always uses the same models:/ URI format.

ALIASES IN ACTION

“Give me the champion model” — and the Registry decides which version that is, not your code.

Reassigning @champion to a new version is a registry decision, never a redeploy.

OSS MLflow vs. Databricks Unity Catalog

Open Source MLflow

- UI and API for versioning and managing models
- Register, tag, describe, and transition versions
- Stage-based lifecycle (Staging → Production)
- Full control, self-hosted on your own infrastructure

Databricks + Unity Catalog

- Centralized governance with access policies
- Cross-workspace registration and reuse
- Model lineage across notebooks, datasets, experiments
- Discovery of production-grade models from a shared catalog

04

MLflow for LLMs, Agents & RAG

The same tracking discipline, applied to prompts, retrieval, and agent behavior.



“It Worked” Isn't Enough for RAG

- 1 Classic ML asks: did the model run?**
In RAG, that's not enough — you also need to know what the prompt was and how retrieval behaved.

- 2 Track the prompt and its versions**
So you can tell exactly what instructions produced a given answer.

- 3 Track embeddings and retrieval configuration**
Retrieval quality drives answer quality — and both change over time.

- 4 Track RAG-specific metrics**
Faithfulness and context recall, tied back to the run that produced them.

THE DIAGNOSTIC QUESTION

If answer quality improves, was it a better model? A different prompt? Better retrieval? New embeddings?

Tracking turns that question into something answerable — instead of a guess.

Built for Production AI, Not Just Notebooks

OTel

OpenTelemetry-native tracing with GenAI semantic conventions — no vendor lock-in

100+

One-line integrations: OpenAI, LangChain, LlamaIndex, DSPy, Pydantic AI & more

95%

Smaller footprint with the lightweight mlflow-tracing SDK for production

See Inside Every AI Decision

- MLflow Tracing captures the inputs, outputs, and metadata of every intermediate step in a request.
- Framework-agnostic: works with any LLM provider or agent framework, so switching stacks doesn't mean losing observability.
- Fully OpenTelemetry-compatible — export to your existing observability stack with no lock-in.
- Navigate traces inside your IDE, notebook, or the MLflow UI — no tab-switching to debug an agent.

One line of code

```
import mlflow

mlflow.openai.autolog()

# swap "openai" for anthropic,
# langchain, dspy, and more
```

Tracing Powers the Whole Lifecycle

1

Build & Debug

Pinpoint exactly where in an agent's execution something went wrong.

2

Human Feedback

Attach reviewer feedback directly to the trace that produced an answer.

3

Evaluation

Feed traces straight into MLflow's evaluation and scoring pipeline.

4

Production Monitoring

Watch live traffic with async logging that doesn't slow the app down.

5

Dataset Collection

Turn real production traces into evaluation datasets for the next iteration.

Flexible Instrumentation, Production-Ready

1

@mlflow.trace decorator, or trace any code block

Manual instrumentation when auto-tracing isn't enough.

2

Multi-threaded & async support

Trace context propagates correctly across threads, async calls, and even distributed services.

3

Sessions, PII masking, and sampling

Group traces by user session, redact sensitive data, and control trace volume in production.

4

Async trace logging

Traces are written in the background so they never add latency to a live request.

EVALUATION & MONITORING

Evaluation-Driven Development: treat every prompt, model, or retrieval change like code — measure before you ship it.

Key Evaluation Capabilities

1

Dataset Management

A centralized “test database” of inputs and ground-truth expectations, at scale.

2

Human Feedback

Capture reviewer judgments and attach them directly to runs and traces.

3

LLM-as-a-Judge

Use LLM judges to score free-form language against hallucination, relevance, and more.

4

Systematic Evaluation

Run the same scorers across every change, instead of eyeballing outputs by hand.

5

Production Monitoring

Continue scoring quality once the agent is live, not just during development.

Evaluation Datasets & Running an Evaluation

- Evaluation Datasets are a centralized repository for test cases, ground-truth expectations, and evaluation data.
- They turn ad-hoc, one-off testing into a repeatable quality-assurance process your whole team can rely on.
- An evaluation run combines a dataset of inputs, the app or model being tested, and a set of scorers or judges.
- Results attach back to the run, so a quality change is always traceable to the change that caused it.

Think of it as...

A test database for your AI application — the single source of truth every evaluation run reads from.

Version, Track, and Reuse Prompts

- Git-inspired, commit-based versioning with side-by-side diff comparison between prompt versions.
- Prompt versions are immutable — strong guarantees for reproducibility.
- Aliases isolate prompt versions from your application code, enabling safe A/B testing and instant rollbacks.
- Fully integrated with Tracing, Evaluation, and Monitoring for end-to-end lineage.

Also included

Automated prompt optimization using data-driven algorithms — no manual trial-and-error required.

Register, Version & Use a Prompt



python

```
prompt = mlflow.genai.register_prompt(  
    name="summarization-prompt",  
    template="Summarize in {{ num_sentences }} sentences:\n{{ sentences }}",  
    commit_message="Initial commit",  
)  
  
# Later, in your app  
prompt = mlflow.genai.load_prompt("prompts:/summarization-prompt/2")  
filled = prompt.format(num_sentences=1, sentences=target_text)
```

Double-curly {{ variables }} fill in at call time — the template itself never changes once created.

Three Prompt Types

1

Text prompts

Simple string templates: “Hello {{ name }}, how are you today?”

2

Chat prompts

A list of role/content messages — system, user, assistant — for chat-style APIs.

3

Jinja2 prompts

Full conditionals and loops ({% if %}, {% for %}) for advanced templating, auto-detected from the syntax.

Model Config & Prompt Caching

- Attach `model_config` to a prompt — model name, temperature, `max_tokens`, `top_p`, and more — so the prompt and its inference settings version together.
- Model config is mutable and can be updated without creating a new prompt version.
- Version-based prompts (`prompts:/name/1`) cache with an infinite TTL by default — they're immutable.
- Alias-based prompts (`prompts:/name@production`) cache for 60 seconds by default, so updates propagate quickly.

Reproducibility, built in

Share a prompt with its recommended model settings across the whole team — no more “which temperature did we use?”

One Endpoint for Every LLM Provider

- A unified interface for OpenAI, Anthropic, and other providers — no need to integrate each one separately.
- API keys live in one secure location, with request/response logging for audit and compliance.
- Traffic splitting for A/B testing, plus automatic fallback chains for high availability.
- Passthrough endpoints forward requests in a provider's native format, so new capabilities are usable immediately.

Zero-downtime updates

Add, remove, or reconfigure endpoints on the fly — without restarting the server or disrupting live applications.

Governance Built Into the Gateway

1

Usage Tracking

Monitor endpoint usage, latency, token consumption, and cost across every provider.

2

Budget Alerts & Limits

Set spending limits and get alerted before costs run away.

3

Guardrails

LLM-powered judges that block or sanitize risky requests and responses.

4

Traffic Routing & Fallbacks

Split traffic across providers, and fail over automatically when one is down.

5

Endpoints

Create, configure, and query model endpoints from one control plane.

6

LLM Playground

Test endpoints interactively with multi-turn chat, sampling parameters, and tools.

05

MLflow vs. Weights & Biases

Both solve experiment tracking — the right choice depends on your workflow.



Side by Side

Point	MLflow	Weights & Biases
Hosting	Self-hosted or on-prem	Cloud-first experience
Privacy	Ideal when data must stay in-house	Best when cloud usage is acceptable
Tracking	Experiments, metrics, artifacts, models	Experiments, metrics, artifacts + rich UI
Registry	Model Registry built in	Model management within the platform
Optimization	Build your own workflows	Strong built-in sweeps & hyperparameter tuning

When to Choose Which

Choose MLflow

- Working with a bank, healthcare org, or any strict data-privacy requirement
- Need full control over infrastructure and hosting
- Want an open, vendor-neutral platform across ML and GenAI
- Building production agents that need tracing + evaluation + registry together

Choose W&B

- Want a fast, cloud-based experience with minimal setup
- Your team leans heavily on sweeps and hyperparameter tuning
- Cloud usage is already acceptable for your data
- You prioritize a polished, ready-made UI over self-hosting

THE REAL QUESTION

It's rarely “which tool is better?” — it's “which tool fits my infrastructure and workflow?”

The Five-Minute Summary

1

MLflow

Organizes and tracks the entire ML lifecycle, from first experiment to production.

2

Experiment Tracking

Know exactly what happened in every experiment — parameters, metrics, artifacts.

3

Model Registry

Know exactly which model version is meant to be used, right now.

4

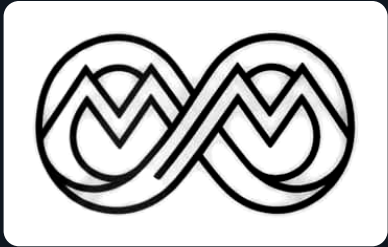
RAG Tracking

Follow prompts, retrieval, and quality metrics — not just a loss curve.

5

MLflow vs. W&B

Choose based on privacy, hosting, workflow, and how much automation you need.



Thank You!

Questions, discussion, and more MLOps content — join us on our channels.

- mlflow.org/docs/latest/genai/ — LLMs & Agents documentation
- mlflow.org/docs/latest/ml/ — Classic ML documentation
- demo.mlflow.org — Try the live public demo
- github.com/mlflow/mlflow — Star the project

